CardDemo (Rust) — Architecture and API

Derived from the Rust source code of the translation

Illia Burov (prepared with Claude Code)

2026-11-07 — version 1.0

Contents

1 Introduction **1**
- 1.1 Purpose . . . 1
- 1.2 How this document was made: derived from the code . . . 2
- 1.3 Design principles . . . 2

2 Workspace and crates **2**

3 Crate APIs **3**
- 3.1 `cobol`: COBOL data semantics . . . 3
- 3.2 `vsam`: data store . . . 4
- 3.3 `cics`: the CICS runtime . . . 4
- 3.4 `online`: the CICS programs . . . 5
- 3.5 `batch`: programs and job streams . . . 6
- 3.6 `cobridge`: the original COBOL programs on the Rust runtime . . . 6
- 3.7 `carddemo`: the executable . . . 7

4 Runtime behaviour **8**
- 4.1 An online transaction . . . 8
- 4.2 A batch job . . . 8
- 4.3 An original COBOL program . . . 8

5 From COBOL to Rust **8**

6 Verification **8**

7 Build, platforms and release **9**

1 Introduction

1.1 Purpose

This document describes the architecture and the programming interfaces (APIs) of the **Rust translation** of CardDemo in `rust/`: the crates and how they depend on each other, the public API of each crate, how an online transaction, a batch job and an original COBOL program run, how the COBOL programs map to Rust code, how data is stored, and how the translation is tested and shipped.

The original application is described in *CardDemo (COBOL) — Architecture and Interfaces* (`architecture-cobol.pdf`), its behaviour in the *Software Requirements Specification* (`requirements.pdf`).

1.2 How this document was made: derived from the code

This document was derived from the Rust source code; it is not a design written before the code. The statements were obtained by reading the workspace in `rust/` — the crate manifests (`Cargo.toml`), the public items of each crate (`pub fn`, `pub struct`, `pub enum` and their doc comments), the module documentation (`//!`), the test suites and the tools (`difftest/`, `cobridge/cicsprep.py`, `tools/`) — partly with small extraction scripts, and checked against the code. The source was taken from the repository at commit `49ee1a5` (7 November 2026), version 1.0.0 of the workspace.

Consequences for the reader:

- **Where this document and the code disagree, the code is right.** For exact signatures and behaviour, the doc comments in the code (`cargo doc --workspace --open`) are the reference; this document gives the overview they do not.
- Signatures are quoted as they are in the code, sometimes shortened (lifetimes and trait bounds left out).
- Design decisions are described as the code shows them; where the code says *why* (in comments, the progress log `docs/progress-log.md` or `CLAUDE.md`), the reason is given.

1.3 Design principles

As the code and `CLAUDE.md` state them:

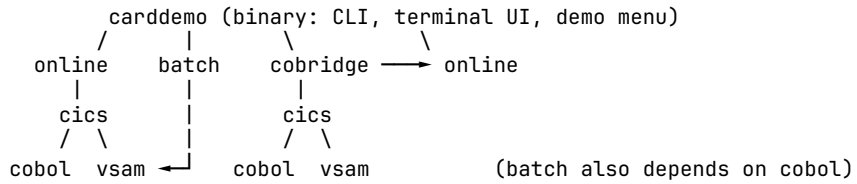
1. **Faithful translation.** The Rust programs produce the same screens, messages, records, reports and statements as the original, including its defects (*KD-nn* in the requirements). A difference from the COBOL is a bug in the Rust code.
2. **One Rust function per COBOL paragraph**, named after it, so each part of the translation can be checked against its original.
3. **The mainframe services are libraries**, not emulated machines: COBOL data semantics, VSAM, CICS and JCL each became a crate with the behaviour the programs rely on.
4. **Proof by comparison.** The original COBOL programs run on GnuCOBOL against the same data, and every batch output and every screen of the scripted sessions is compared with the Rust result.
5. **One program, no runtime.** The result is one executable (`carddemo`) with an embedded SQLite data store; it needs no mainframe, CICS, VSAM or COBOL runtime and runs on Linux and Windows (NFR-09).

2 Workspace and crates

The Cargo workspace `rust/` has seven crates. The lower ones know nothing of the upper ones.

Crate	Replaces on the mainframe	Depends on (workspace)	Main external dependencies
`cobol`	COBOL data semantics: PICTURE, USAGE, MOVE, STRING, editing, intrinsic functions, EBCDIC	—	`rust_decimal`, `chrono`
`vsam`	VSAM KSDS and alternate indexes, catalog	—	`rusqlite` (bundled SQLite)
`cics`	CICS: tasks, EIB, COMMAREA, XCTL/RETURN, file control, BMS, TD queue	`cobol`, `vsam`	`chrono`
`online`	The 17 online programs	`cobol`, `vsam`, `cics`	`rust_decimal`, `chrono`

Crate	Replaces on the mainframe	Depends on (workspace)	Main external dependencies
`batch`	The batch programs, the JCL job streams, IDCAMS/SORT/QSAM	`cobol`, `vsam`	`rust_decimal`, `anyhow`
`cobridge`	Runs the **original** online COBOL programs on the `cics` runtime (GnuCOBOL, Linux)	`cobol`, `vsam`, `cics`, `online`	`libc` (dlopen of libcob)
`carddemo`	The executable: terminal UI, job runner, demo menu	all of the above	`clap`, `ratatui`, `crossterm`, `libc`



Minimum Rust version 1.88 (`rust-version` in the workspace, checked: the locked dependencies need it).

3 Crate APIs

3.1 `cobol`: COBOL data semantics

Module	API	Purpose
`numeric`	`Dec` (= `rust_decimal::Decimal`), `Pic { digits, scale, signed, usage }`, `Usage`	The picture of a numeric item and its storage (DISPLAY, COMP-3, COMP)
	`encode(v, pic, out)`, `decode(b, pic) → Dec`, `fit(v, pic) → Dec`	Store and load a numeric item; truncation rules of a receiving item
	`is_numeric(b, signed) → bool`, `to_i64(d)`	Class test `IS NUMERIC`; integer part
`text`	`mv`, `mv_just_right`, `string_into`, `until`, `inspect_converting`	`MOVE` (left-justify, pad, truncate), `JUSTIFIED RIGHT`, `STRING ... DELIMITED BY`, `INSPECT ... CONVERTING`
	`is_spaces`, `is_low`, `is_zeros`, `is_blank`, `trim`, `upper`, `lower`, `s`	Figurative-constant tests and intrinsic functions; `s` gives a printable `String`
	`numval`, `numval_c`, `test_numval`, `test_numval_c`; `LOW`, `HIGH`	`FUNCTION NUMVAL(-C)`, `TEST-NUMVAL(-C)`; LOW-VALUE and HIGH-VALUE
`edit`	`edit(v, pic) → String`	Edited pictures (`-ZZZ,ZZ9.99`, `+9(8).99`, ...)
`field`	`get_n`, `set_n`, `set_edited`	Numeric access to a byte range
`ebcdic`	`to_ascii`, `to_ebcdic`, `EBCDIC_TO_LATIN1`, `LATIN1_TO_EBCDIC`	Code page 037

Module	API	Purpose
`clock`	`now()`, `fixed_now()`	The clock behind `FUNCTION CURRENT-DATE` and `ASKTIME`; `CARDDEMO_NOW` freezes it for tests
`copybooks`	One struct per record (generated)	See below

Records. `copybooks.rs` is generated from the copybooks by `tools/gen_records.py` (`tools/regen-records.sh`). Each 01-level record becomes a struct holding its bytes (`pub buf: [u8; LEN]`) with:

- `LEN`, `new()` (all spaces), `from_bytes(&[u8])`;
- for every field `x`: `x()` → `&[u8]` and `set_x(&[u8])` (a `MOVE`), and for numeric fields also `x_n()` → `Dec`, `set_x_n(Dec)` and the constant `X_PIC`.

The 21 records: `AccountRecord` (300), `CardRecord` (150), `CardXrefRecord` (50), `CustomerRecord` (500), `TranRecord` (350), `DalytranRecord` (350), `TranCatBalRecord`, `DisGroupRecord` (50), `TranTypeRecord`, `TranCatRecord` (60), `SecUserData` (80), `TrnxRecord` (350), `ExportRecord` (500), `CarddemoCommarea` (160) and the seven report lines of `CVTRA07Y`. Storage without a `VALUE` clause must start as binary zeros: `Record::from_bytes(&[0; Record::LEN])`, not `new()`.

3.2 `vsam`: data store

```
pub struct Vsam { /* SQLite connection */ }
impl Vsam {
    pub fn open(path) → rusqlite::Result<Vsam>;     pub fn open_in_memory() → rusqlite::Result<Vsam>;
    pub fn read(&self, file, key) → VResult<Vec<u8>>;   pub fn read_ex(&self, file, key) →
    ↪  VResult<(Vec<u8>, bool)>;
    pub fn write(&self, file, rec) → VResult<()>;       pub fn rewrite(&self, file, rec) → VResult<()>;
    pub fn delete(&self, file, key) → VResult<()>;
    pub fn start_browse(&self, file, key, gteq) → VResult<Browse>;
    pub fn read_next(&self, &mut Browse) → VResult<Vec<u8>>;  pub fn read_prev(&self, &mut Browse) →
    ↪  VResult<Vec<u8>>;
    pub fn begin/commit/rollback/syncpoint/syncpoint_rollback(&self) → VResult<()>;
    pub fn clear(&self, dataset); pub fn unload(&self, dataset) → VResult<Vec<Vec<u8>>>; pub fn
    ↪  count(&self, dataset);
}
pub enum Resp { NotFnd, DupRec, DupKey, InvReq, IoErr, NotOpen, EndFile, FileNotFound, ... }  // .code() =
↪  EIBRESP
pub type VResult<T> = Result<T, Resp>;
```

- `file` is a CICS file name (`ACCTDAT`, `CXACAIX`) or a data set name; the catalog (`CATALOG`, `CICS_FILES`) resolves it to a cluster or an alternate-index path. `read_ex` also reports a duplicate alternate key (CICS `DUPKEY`).
- **Storage**: every KSDS is a SQLite table `k_<cluster>(k BLOB PRIMARY KEY, r BLOB)`, every alternate index a table `a_<cluster>_<aix>(ak, k)` maintained on each write. Keys are stored **in EBCDIC**, so SQLite's byte order is the mainframe's key order; records are stored as the programs see them.
- **Browses** follow CICS: the first `READPREV` after `STARTBR` returns the record the browse is positioned on; a key of HIGH-VALUES positions after the last record.
- **Units of work**: each online task and each batch step runs in one SQLite transaction (`begin`/`commit`; `syncpoint` commits and begins anew). A store held by another process is reported, not waited on forever.

3.3 `cics`: the CICS runtime

```
pub type Program = fn(&mut Task) → Cr;            // a CICS program
pub type Cr<T = ()> = Result<T, Transfer>;        // Err = the program leaves (XCTL, RETURN, ABEND)
pub enum Transfer { Xctl { program, commarea }, Return { transid, commarea }, Abend(String) }
pub enum Aid { Enter, Clear, Pa1, Pa2, Pf(u8) }    // EIBAID

pub struct Region { pub vsam, pub maps: MapCatalog, pub programs, pub transactions,
                    pub applid, pub sysid, pub on_job: Option<JobReader> }
impl Region { pub fn run(&self, sess: &mut Session, transid: Option<&str>, aid: Aid) }
```

```
pub struct Session { pub screen: Screen, pub next_transid: Option<String>, pub commarea: Vec<u8>, pub log:
↪  Vec<String> }
```

`Task` is what a program sees of CICS (`EIB*` fields, COMMAREA) and offers the commands:

Task method	CICS command
`xctl(program, commarea) → Transfer`, `ret(transid, commarea) → Transfer`, `abend(code) → Transfer`	`XCTL`, `RETURN`, `ABEND` (returned as `Err(...)`)
`map(name) → Map`, `receive_map(name) → Map`, `send_map(&map, erase)`, `send_text(bytes)`	`RECEIVE MAP`, `SEND MAP`, `SEND TEXT`
`read(file, key, update)`, `read_dup`, `write`, `rewrite`, `delete`	file control
`startbr(file, key, gteq) → Browse`, `readnext`, `readprev`	browsing
`syncpoint()`, `syncpoint_rollback()`, `writeq_td(queue, data)`, `program_exists(name)`	`SYNCPOINT`, `WRITEQ TD`, `INQUIRE PROGRAM`

- **XCTL and RETURN end a program at once**: they are values of `Err(Transfer)` that any paragraph returns with `?`. `SEND MAP` does not end the program; the last screen sent is what the terminal shows.
- **BMS**: `bms::parse_mapset` reads the original BMS sources (`cobol/app/bms`, compiled into the binary by `MapCatalog::builtin()`). A `Map` is the symbolic map (`xxxL`, `xxxA`, `xxxC`, `xxxI`/`xxxO` of each field, input and output sharing storage as the COBOL `REDEFINES` do); `Screen` is what a 3270 holds (fields with attributes, colours, the cursor) and offers `type_into`, `field_at` and `render` for the UI and the tests.
- **TD queue** `JOBS`: `writeq_td("JOBS", card)` collects card images; at the end of the task the deck goes to `Region::on_job` (the internal reader).

3.4 `online`: the CICS programs

```
pub fn region(vsam: Vsam) → Region;                    // all base programs and maps
pub fn programs() → HashMap<&str, Program>;            // program table (PPT)
pub fn transactions() → HashMap<&str, &str>;            // transaction table (PCT), 17 entries
```

Module	Programs (each `pub fn <name>(t: &mut Task) → Cr`)	Shared code
`signon`	`cosgn00c`, `comen01c`, `coadm01c`	menu tables `COMEN02Y`, `COADM02Y`
`accounts`	`coactvwc`, `coactupc`	
`cards`	`cocrdlic`, `cocrdslc`, `cocrdupc`	`store_pfkey` (`CSSTRPFY`), `file_error`, `pf3_back`, commarea split/join
`transactions`	`cotrn00c`, `cotrn01c`, `cotrn02c`, `cobil00c`, `corpt00c`	`report_jcl` (the JCL deck `CORPT00C` writes to `JOBS`)
`users`	`cousr00c`, `cousr01c`, `cousr02c`, `cousr03c`	
`csutldtc`	`csutldtc` (the date validation routine)	`CEEDAYS` behaviour
`common`, `lookups`	—	titles, messages, date/time, POPULATE-HEADER-INFO, commarea helpers (`COTTL01Y`, `CSMSG01Y`, `CSDAT01Y`, `COCOM01Y`); phone area codes and state/ZIP tables (`CSLKPCDY`)

Within each program, the paragraphs are private functions named after the COBOL paragraphs.

3.5 `batch`: programs and job streams

```rust
pub fn run_job(env: &Env, job: &str, opts: &JobOpts) -> Result<Vec<StepLog>, (Vec<StepLog>, Abend)>;
pub struct Env<'a> { pub vsam: &'a Vsam, pub root: PathBuf, pub seed: PathBuf, pub echo: bool }
pub struct JobOpts { pub intcalc_parm, pub report_start, pub report_end, pub dalytran: Option<PathBuf> }
pub const JOBS: &[(&str, &str)];                                  // the 27 jobs with their purpose

pub struct StepCtx<'a> { pub vsam, pub dds: HashMap<String, Dd>, pub parm: String, pub sysout, pub echo }
pub enum Dd { Vsam(String), File(PathBuf, Format), Instream(Vec<Vec<u8>>) }   // what a DD points at
pub type StepResult = Result<i32, Abend>;                        // return code, or abend (CEE3ABD code)
```

- **Jobs** (`jobs.rs`): each JCL member is a `match` arm that runs its steps in order: `env.step(...)` for a program with its DD statements, `env.util(...)` for IDCAMS, SORT, IEBGENER, IEFBR14. `StepLog` records step, program and return code or abend; a step that abends ends the job (the remaining steps do not run). The JCL's `COND` parameters are not evaluated: return codes are recorded and reported (`MAXCC`) only.
- **Programs**: `fn(&mut StepCtx) → StepResult`. A program opens its files by DD name (`ctx.input("DALYTRAN")`, `ctx.output(...)`, `ctx.vsam_name("ACCTFILE")`), displays to SYSOUT (`ctx.display`), and abends through `ctx::abend` (`9999-ABEND-PROGRAM`).
- **Data sets**: VSAM clusters are in the store; sequential data sets and GDG generations are files in `<root>/datasets/` (`AWS.M2.CARDDEMO.X`, generations `X.G0001V00`, ...). `seqfile` reads and writes fixed-length records as line-sequential ASCII, raw or EBCDIC; `repro` does IDCAMS `REPRO` and the initial load (`reset`, `sample_store`, `sample_seed_dir`).

Module	COBOL programs
`cbtrn02c`, `cbact04c`, `cbtrn03c`	`CBTRN02C` (POSTTRAN), `CBACT04C` (INTCALC), `CBTRN03C` (TRANREPT)
`cbstm03a`	`CBSTM03A` and `CBSTM03B` (CREASTMT)
`export`	`CBEXPORT`, `CBIMPORT`
`readers`	`CBACT01C` (with the assembler `COBDATFT`), `CBACT02C`, `CBACT03C`, `CBCUS01C`, `CBTRN01C` (job `DALYVRFY`, which has no JCL in the original)
`jobs`	the JCL job streams; `COBSWAIT`/`MVSWAIT` as job `WAITSTEP`

3.6 `cobridge`: the original COBOL programs on the Rust runtime

```rust
pub fn region(vsam: Vsam) -> Region;            // like online::region, but every program is the COBOL one
pub fn start() -> Result<(), String>;           // load libcob, cob_init (once); also done by the first call
pub fn check() -> Result<(), String>;           // modules built? libcob loadable?
pub fn modules_dir() -> PathBuf;               // CARDDEMO_COBOL_MODULES, cobol-modules next to the program, build dir
pub fn set_sysout(path) -> io::Result<()>;      // where DISPLAY output and libcob messages go
pub fn sysout_path() -> Option<PathBuf>;  pub fn write_sysout(&str) -> bool;  pub fn send_to_sysout(fd) -> io::Result<bool>;
pub fn in_cobol() -> bool;  pub fn in_cdcics() -> bool;  pub fn note_panic() -> bool;   // for the UI's hooks
```

How it works. `cobridge/cicsprep.py` translates each original program, as IBM's CICS translator does, replacing every `EXEC CICS ... END-EXEC` with

```
CALL EIB-CDCICS USING DFHEIBLK
     BY CONTENT 'Dn' 'descriptor piece 1' ... 'descriptor piece n'
     <argument 1> <argument 2> ...
IF RETURN-CODE NOT = 0 GOBACK END-IF
```

and compiles it with GnuCOBOL into a module (`.so`). `EIB-CDCICS` is a procedure pointer that the bridge stores in the EIB, so the Rust side needs no exported symbols. The descriptor names the command and its options (`READ DATASET=1 RIDFLD=2 INTO=3 ... RESP`); the bridge's entry point `cdcics` reads the arguments through GnuCOBOL's C API, carries out the command on the `cics` runtime (`SEND`, `RECEIVE`, `RETURN`,

XCTL, ABEND, HANDLE, READ, WRITE, REWRITE, DELETE, STARTBR, READNEXT, READPREV, ENDBR, SYNCPOINT, WRITEQ, INQUIRE, ASKTIME, FORMATTIME, ASSIGN), sets EIBRESP/EIBRESP2 and returns 0 (continue), *n* (go to the *n*-th HANDLE label) or 99 (leave the program).

EIB field (bridge's DFHEIBLK)	Offset	Length
EIBTIME, EIBDATE (packed)	0, 4	4, 4
EIBTRNID, EIBTRMID	8, 16	4, 4
EIBCALEN	24	2
EIBAID	26	1
EIBRESP, EIBRESP2	59, 63	4, 4
EIB-CDCICS (procedure pointer)	67	8

- run (the Program of every COBOL transaction) builds the EIB, passes the commarea in a zero-filled 32 KB area (EIBCALEN keeps the real length; KD-16), calls the module with cob_call, and cancels it afterwards: CICS gives each task fresh WORKING-STORAGE.
- libcob is loaded at run time (dlopen; CARDDEMO_LIBCOB names the exact library), so every build works without GnuCOBOL. On Windows the bridge builds and reports that it runs on Linux only.
- **Output**: set_sysout points the C library's stdout/stderr streams (which libcob writes to) at a file; the process's descriptors stay as they are, so the terminal UI and the test harness are not disturbed.
- **Failures**: a panic inside a command is caught and ends the task with abend ASRA (note_panic tells a panic hook so); in_cobol tells an exit handler that libcob is ending the process.
- Supporting files: the IBM copybooks DFHAID, DFHBMSCA, DFHATTR and DFHEIBLK in cobridge/copy/, and CEEDAYS in C (cobridge/ceedays.c).

3.7 carddemo: the executable

Command	Function
(none), demo	The demo menu: online screens, nightly batch, transaction report, output-file viewer, reset, original COBOL screens (Linux)
online [--tran X] [--cobol]	The online system in the terminal; --cobol runs the original programs (cobridge)
job NAME [--parm, --start, --end, --dalytran], batch [JOBS...], jobs	Run one job, a sequence (default: the nightly batch), list the jobs
reset, unload DIR	Load the sample data; unload every cluster to fixed-length files
script STEPS...	Print the screen after scripted keystrokes (documentation)

Options: --home (data directory; the menu keeps its data next to the program), --seed (sample data, found in the repository or in data/EBCDIC next to the program).

The terminal (term.rs): raw mode and the alternate screen are set up like ratatui::try_init() but with the program's own panic hook, and taken off **exactly once on every way out**: the normal end or an error (a guard), a panic (the hook; except a panic the COBOL bridge catches, whose message goes to the sysout file), a process exit (an exit handler, which also names the sysout file when libcob ended the process), and every signal whose default action ends the process (handlers that restore the terminal with async-signal-safe calls and pass the signal on, re-installed in front of libcob's on every start of the UI). With --cobol, descriptor 2 points at the sysout file while the UI is on.

4 Runtime behaviour

4.1 An online transaction

1. The terminal UI (or a test) calls `Region::run(session, transid, aid)` for a key press.
2. The transaction is the one given, else the session's pending `RETURN TRANSID`, else `CC00`; the PCT gives the program. An unknown transaction shows `DFHAC2001`.
3. A unit of work begins (`vsam.begin`); a `Task` is built with the commarea, `EIBCALEN`, `EIBAID`, the terminal's screen.
4. The program runs. `XCTL` starts the next program in the same task with the new commarea; `RETURN` ends the task and stores `TRANSID` and commarea in the session; an abend shows `DFHAC2206` and backs out the updates.
5. The unit of work is committed (or rolled back); the last screen sent becomes the session's screen; a JCL deck written to `JOBS` goes to `on_job` (the terminal UI runs the submitted report job).

4.2 A batch job

`run_job` resolves the job's DD statements to `Dd` values (cluster, file of a data set or GDG generation, in-stream cards), runs each step in its own unit of work and records the step's return code or abend; an abend ends the job. The nightly batch is `TRANBKP`, `POSTTRAN`, `INTCALC`, `TRANBKP`, `COMBTRAN`, `CREASTMT`.

4.3 An original COBOL program

`cobridge::run` (one per COBOL transaction) → `cob_call` of the module → at each `EXEC CICS`, a call to `cdcics` → the same `Task` methods the Rust programs use → return to the COBOL program → at `RETURN`/`XCTL`, return code 99 and `GOBACK` → `cob_cancel`. So both implementations share screens, files, transactions and the data store, which is what makes the screen-by-screen comparison possible.

5 From COBOL to Rust

Original	Rust
Online program `COxxxxxC`	`online::<module>::coxxxxxc`, paragraphs as private functions
Batch program `CBxxxxxC`	`batch::<module>::run` (or named function), paragraphs as functions
Copybook record	`cobol::copybooks::<Record>` (generated)
BMS mapset	parsed at build time from `cobol/app/bms` (`cics::bms`)
`EXEC CICS` command	`Task` method; XCTL/RETURN/ABEND as `Err(Transfer)`
JCL job	`batch::jobs::run_job` arm; IDCAMS, SORT, IEBGENER, IEFBR14 as step utilities
VSAM cluster / AIX / path	SQLite tables in the `vsam` store, catalog in `vsam::catalog`
Sequential data set / GDG	file in `<root>/datasets/` (`.G0001V00` generations)
`CEE3ABD`, `CEEDAYS`, `COBDATFT`, `MVSWAIT`	`batch::ctx::abend`, `online::csutldtc`, `batch::readers::cobdatft`, job `WAITSTEP`
CSD (PCT/PPT, files, TD queue)	`online::transactions()`, `online::programs()`, `vsam::CICS_FILES`, `Region::on_job`

6 Verification

Level	Where	What it checks
Unit tests	each crate (`#[cfg(test)]`)	data semantics, VSAM, BMS parsing, panic handling, sysout
Scripted online sessions	`crates/online/tests/` (manual suite 40 cases, users, cards and accounts, transactions, sign-on, review fixes, paragraph coverage, multi-session)	every screen of each step
Batch jobs	`crates/batch/tests/`	the job streams on the sample data
Batch comparison	`difftest/run.py`	the original batch programs (GnuCOBOL) and Rust on the same input: every output data set, report, return code and SYSOUT line (40 outputs, identical)
Online comparison	`difftest/online.py`	the scripted sessions on the Rust programs and on the original COBOL programs (`cobridge`): every screen, field, attribute and cursor (399 screens, identical)
Paragraph coverage	`difftest/coverage.py`	which paragraphs of the original programs the two comparisons enter (GnuCOBOL `-ftrace`): 551 of 599 (92%); the script fails if one of the others has no stated reason
Differential fuzzing	`difftest/fuzz.py`, `crates/online/tests/fuzz_sessions.rs`	random daily transaction files through the whole batch comparison, and random keystroke sessions through the online comparison, seeded and repeatable
Continuous integration	`.github/workflows/ci.yml`	on every push: Linux build, clippy and tests, Windows tests and MinGW cross build, all the comparisons above, a few fuzzing seeds, the manual suite
Terminal behaviour	`crates/carddemo/tests/terminal.rs`	the real binary in a pseudo-terminal: quit, kill, failed setup, stack overflow, libcob failures, the demo menu in the release layout
Manual test suite	`docs/tests/`	42 cases traced to the requirements, executed by script

7 Build, platforms and release

- `cargo build --release` builds everything; GnuCOBOL is needed only to compile and run the original programs (`cobridge/cicsprep.py`, `difftest/`).
- Linux and Windows (NFR-09). The Windows build (`x86_64-pc-windows-gnu`, MinGW) embeds an icon and version information (`crates/carddemo/build.rs`); the COBOL bridge is Linux-only.

- `tools/package-release.sh` builds the downloads: a Windows zip and a Linux tar.gz with the stripped program, the sample data and the quick-start guide, the Linux one also with the compiled COBOL modules.
- `tools/regen-records.sh` regenerates `copybooks.rs` from the copybooks.